LABORATORY RECORD

Python Programming and Data Structures

Experiments 1 to 13

Exp. No.	Title of the Experiment
1	Real-Time Applications of Built-in and User-Defined Data Structures
2	Sets and Set Operations
3	Lists and List Operations
4	Strings and String Methods
5	Exception Handling
6	Recursion
7	Abstract Data Types (ADT) and User-Defined Types (UDT)
8	Stack Data Structure and its Operations
9	Queue Data Structure and its Operations
10	Singly Linked List and its Operations
11	Doubly Linked List and its Operations
12	Tree Data Structure and its Operations
13	Tree Traversals

Name: _____

Register No.: _____

Class / Section: _____

EXPERIMENT 1

Real-Time Applications of Built-in and User-Defined Data Structures

AIM

To explore and document the real-time applications of built-in data structures and user-defined data structures.

PROCEDURE

1. Start.
2. Identify the built-in data structures available in Python: list, tuple, set, dictionary and string.
3. Identify the commonly used user-defined data structures: stack, queue, linked list, tree and graph.
4. Note the key property of each structure (ordering, mutability, uniqueness, access pattern).
5. Study the real-time software and systems in which each structure is used.
6. Document the findings in tabular form and compare built-in and user-defined structures.
7. Stop.

OBSERVATION

Table 1: Built-in data structures and their real-time applications

Data Structure	Key Property	Real-Time Applications
List	Ordered, mutable, allows duplicates	Playlist of songs in a music app; to-do list; items in a shopping cart; storing a series of sensor readings.
Tuple	Ordered, immutable	GPS coordinates (latitude, longitude); RGB colour values; a row returned by a database query; fixed configuration values.
Set	Unordered, unique elements, fast membership test	Counting unique visitors of a website; finding mutual friends (intersection); removing duplicate records; tag management.
Dictionary	Key-value pairs, O(1) average lookup	Fetching a user profile by ID; JSON data in REST APIs; caching (memoization); word-frequency counting; application settings.
String	Immutable sequence of characters	Username and password validation; search queries; chat messages; log file parsing; URL handling.

Table 2: User-defined data structures and their real-time applications

Data Structure	Key Property	Real-Time Applications
Stack	LIFO (Last In, First Out)	Undo/redo in text and image editors; browser back button; function call management; parentheses and expression checking in compilers.
Queue	FIFO (First In, First Out)	Printer spooling; CPU and task scheduling; customer-support ticket handling; order-processing message queues.
Singly Linked List	Nodes linked in one direction; dynamic size	Hash-table chaining; free-memory block management; image slideshow; building stacks and queues.
Doubly Linked List	Nodes linked in both directions	Browser forward/back history; music player next/previous; LRU cache; cursor movement in a text editor.

Data Structure	Key Property	Real-Time Applications
Tree	Hierarchical; BST gives ordered data	File-system folders; HTML DOM; database indexes (B-tree); auto-complete (Trie); organisation charts.
Graph	Vertices connected by edges	Social networks; shortest path in map applications; network routing; recommendation systems.

Table 3: Built-in versus user-defined data structures

Aspect	Built-in	User-Defined
Availability	Provided by Python; no coding needed	Must be designed and coded by the programmer
Performance	Implemented in C and highly optimised	Depends on the implementation; usually slower in pure Python
Flexibility	Fixed behaviour	Behaviour and operations can be customised (e.g. restricting access to one end)
Typical use	General storage, lookup and processing of data	Problem-specific needs such as LIFO/FIFO order, hierarchy and relationships
Examples	list, tuple, set, dict, str	Stack, Queue, Linked List, Tree, Graph

- Built-in structures are ready to use and optimised, so they are the first choice for general-purpose tasks.
- User-defined structures are needed when the problem demands a specific behaviour that the built-in types do not enforce, such as LIFO order, FIFO order or a hierarchy.
- The choice of structure depends on the access pattern: lookup by key suggests a dictionary, uniqueness a set, LIFO a stack, FIFO a queue and hierarchy a tree.
- Real systems often combine structures; for example, an LRU cache uses a dictionary together with a doubly linked list.

RESULT

Thus, the real-time applications of built-in and user-defined data structures were explored and documented.

VIVA QUESTIONS

1. **Name the built-in data structures in Python.**
   Ans: List, tuple, set, dictionary and string (frozenset, bytes and bytearray are also built-in).
2. **Why is a dictionary used for caching?**
   Ans: It provides O(1) average-time lookup by key, so stored results can be retrieved instantly.
3. **Why are tuples used for fixed data such as coordinates?**
   Ans: They are immutable, hashable and memory-efficient, which protects the data from accidental change.
4. **When is a user-defined data structure required?**
   Ans: When the problem needs behaviour that built-in types do not provide, such as restricted access (stack, queue) or links between nodes (list, tree, graph).
5. **Which data structure is used to implement undo and redo, and why?**
   Ans: Stacks, because the most recent action must be reversed first (LIFO).
6. **Which structures together form an LRU cache?**
   Ans: A dictionary for fast lookup and a doubly linked list to keep items in order of recent use.

EXPERIMENT 2

Sets and Set Operations

AIM

To write and execute Python programs to understand the concept of Sets and its operations.

ALGORITHM

1. Start.
2. Create two sets A and B using curly braces; show that duplicates are removed automatically.
3. Perform union (|), intersection (&), difference (-) and symmetric difference (^) on A and B.
4. Test the subset (<=) and disjoint (isdisjoint) relations.
5. Modify set A using add() and discard().
6. Display the result of every operation.
7. Stop.

PROGRAM

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

print("Duplicates removed:", {1, 2, 2, 3, 3, 3})
print("Set A:", A)
print("Set B:", B)
print("Union (A | B):", A | B)
print("Intersection (A & B):", A & B)
print("Difference (A - B):", A - B)
print("Symmetric Difference (A ^ B):", A ^ B)
print("Is {1, 2} a subset of A?", {1, 2} <= A)
print("Is A disjoint from {9, 10}?", A.isdisjoint({9, 10}))

A.add(10)
A.discard(1)
print("After add(10) and discard(1):", A)
```

OUTPUT

```
Duplicates removed: {1, 2, 3}
Set A: {1, 2, 3, 4, 5}
Set B: {4, 5, 6, 7, 8}
Union (A | B): {1, 2, 3, 4, 5, 6, 7, 8}
Intersection (A & B): {4, 5}
Difference (A - B): {1, 2, 3}
Symmetric Difference (A ^ B): {1, 2, 3, 6, 7, 8}
Is {1, 2} a subset of A? True
Is A disjoint from {9, 10}? True
After add(10) and discard(1): {2, 3, 4, 5, 10}
```

OBSERVATION

- A set stores only unique elements; the duplicate values in {1, 2, 2, 3, 3, 3} were removed automatically.
- Union contains every element of A and B, while the intersection contains only the common elements 4 and 5.
- Union, intersection, difference and symmetric difference return new sets and do not modify A or B; add() and discard() modify A in place.
- Sets are unordered and cannot be indexed.

RESULT

Thus, the Python program for Sets was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is a set in Python?**
   Ans: An unordered, mutable collection of unique and hashable elements.
2. **What is the difference between remove() and discard()?**
   Ans: remove() raises a KeyError if the element is absent; discard() does nothing in that case.
3. **How do you create an empty set?**
   Ans: Using set(). The expression {} creates an empty dictionary, not a set.
4. **What is the difference between a set and a frozenset?**
   Ans: A frozenset is immutable and hashable, so it can be used as a dictionary key or as an element of another set.
5. **What is the average time complexity of a membership test in a set?**
   Ans: O(1), because sets are implemented using hash tables.

EXPERIMENT 3

Lists and List Operations

AIM

To write and execute Python programs to understand the concept of Lists and its operations.

ALGORITHM

1. Start.
2. Create a list of integers.
3. Add elements using append() and insert().
4. Delete elements using remove() and pop().
5. Arrange the list using sort() and reverse().
6. Search using index() and extract a portion using slicing.
7. Find the length, maximum and sum, and build a new list using list comprehension.
8. Display the list after every operation.
9. Stop.

PROGRAM

```
nums = [40, 10, 30, 20]
print("Original list:", nums)

nums.append(50)
nums.insert(1, 15)
print("After append(50) and insert(1, 15):", nums)

nums.remove(30)
popped = nums.pop()
print("After remove(30) and pop():", nums, "| popped:", popped)

nums.sort()
print("Sorted:", nums)
nums.reverse()
print("Reversed:", nums)

print("Index of 15:", nums.index(15))
print("Slice nums[1:3]:", nums[1:3])
print("Length:", len(nums), "| Max:", max(nums), "| Sum:", sum(nums))

squares = [x * x for x in range(1, 6)]
print("List comprehension (squares):", squares)
```

OUTPUT

```
Original list: [40, 10, 30, 20]
After append(50) and insert(1, 15): [40, 15, 10, 30, 20, 50]
After remove(30) and pop(): [40, 15, 10, 20] | popped: 50
Sorted: [10, 15, 20, 40]
Reversed: [40, 20, 15, 10]
Index of 15: 2
Slice nums[1:3]: [20, 15]
Length: 4 | Max: 40 | Sum: 85
List comprehension (squares): [1, 4, 9, 16, 25]
```

OBSERVATION

- Lists are ordered and mutable; the original list changes after append(), insert(), remove(), pop(), sort() and reverse().
- append() adds an element at the end, whereas insert() places it at a given index and shifts the later elements.

- Slicing returns a new list and leaves the original list unchanged.
- The sort() method sorts in place and returns None.

RESULT

Thus, the Python program for Lists was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is the difference between a list and a tuple?**
   Ans: A list is mutable; a tuple is immutable.
2. **What is the difference between append() and extend()?**
   Ans: append() adds its argument as a single element; extend() adds each element of an iterable.
3. **What is the difference between sort() and sorted()?**
   Ans: sort() sorts the list in place and returns None; sorted() returns a new sorted list and keeps the original.
4. **What is the difference between a shallow copy and a deep copy?**
   Ans: A shallow copy duplicates only the outer list, so nested objects are shared; a deep copy duplicates nested objects as well.
5. **What is a list comprehension?**
   Ans: A concise syntax for creating a list from an iterable, e.g. [x * x for x in range(5)].

EXPERIMENT 4

Strings and String Methods

AIM

To write and execute Python programs to understand the concept of Strings and its operations.

ALGORITHM

1. Start.
2. Store a string containing leading and trailing spaces.
3. Apply strip(), upper(), lower() and title() to change the form of the string.
4. Apply replace(), split() and join() to modify and rebuild the string.
5. Apply find(), count() and startswith() to search inside the string.
6. Reverse the string using slicing and test it with isalpha() and isdigit().
7. Display the result of each method.
8. Stop.

PROGRAM

```
text = "  Python Programming Lab  "
s = text.strip()

print("strip():", s)
print("upper():", s.upper())
print("lower():", s.lower())
print("title():", s.title())
print("replace():", s.replace("Lab", "Laboratory"))
print("split():", s.split())
print("find('Pro'):", s.find("Pro"))
print("count('g'):", s.count("g"))
print("startswith('Py'):", s.startswith("Py"))
print("join():", "-".join(s.split()))
print("Reversed:", s[::-1])
print("'abc'.isalpha():", "abc".isalpha(), "| '123'.isdigit():", "123".isdigit())
print("Original text unchanged:", repr(text))
```

OUTPUT

```
strip(): Python Programming Lab
upper(): PYTHON PROGRAMMING LAB
lower(): python programming lab
title(): Python Programming Lab
replace(): Python Programming Laboratory
split(): ['Python', 'Programming', 'Lab']
find('Pro'): 7
count('g'): 2
startswith('Py'): True
join(): Python-Programming-Lab
Reversed: baL gnimmargorP nohtyP
'abc'.isalpha(): True | '123'.isdigit(): True
Original text unchanged: '  Python Programming Lab  '
```

OBSERVATION

- Strings are immutable; every method returns a new string and the original string is unchanged (confirmed by the last output line).
- find() returns the starting index of the first match (or -1 if the substring is absent).
- split() converts a string into a list of words and join() converts it back into a string.
- Slicing with [::-1] reverses the string.

RESULT

Thus, the Python program for Strings was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **Are strings mutable in Python?**
   Ans: No. Strings are immutable; operations create new string objects.
2. **What is the difference between find() and index()?**
   Ans: find() returns -1 when the substring is not found; index() raises a ValueError.
3. **What do split() and join() do?**
   Ans: split() breaks a string into a list using a separator; join() combines the elements of an iterable into one string.
4. **How can a palindrome be checked in Python?**
   Ans: Compare the string with its reverse: s == s[::-1].
5. **What is the difference between a single-quoted, double-quoted and triple-quoted string?**
   Ans: Single and double quotes are equivalent for one-line strings; triple quotes allow multi-line strings and docstrings.

EXPERIMENT 5

Exception Handling

AIM

To write and execute Python programs to understand the concept of Exception Handling and its operations.

ALGORITHM

1. Start.
2. Define a function that divides two numbers inside a try block.
3. Write except blocks for ZeroDivisionError and TypeError, an else block for successful execution and a finally block for clean-up.
4. Call the function with valid, zero-divisor and wrong-type arguments.
5. Catch a built-in ValueError raised by int('abc').
6. Define a user-defined exception class and raise it using the raise keyword; catch it in a try-except block.
7. Display the messages.
8. Stop.

PROGRAM

```
def divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero is not allowed.")
    except TypeError:
        print("Error: Both operands must be numbers.")
    else:
        print(f"Result: {a} / {b} = {result}")
    finally:
        print("Execution of divide() completed.\n")

class NegativeAgeError(Exception):
    pass

def set_age(age):
    if age < 0:
        raise NegativeAgeError("Age cannot be negative.")
    return age

divide(10, 2)
divide(10, 0)
divide(10, "a")

try:
    int("abc")
except ValueError as e:
    print("ValueError caught:", e)

try:
    set_age(-5)
except NegativeAgeError as e:
    print("Custom exception caught:", e)
```

OUTPUT

```
Result: 10 / 2 = 5.0
Execution of divide() completed.

Error: Division by zero is not allowed.
Execution of divide() completed.

Error: Both operands must be numbers.
Execution of divide() completed.

ValueError caught: invalid literal for int() with base 10: 'abc'
Custom exception caught: Age cannot be negative.
```

OBSERVATION

- The else block ran only for divide(10, 2), where no exception occurred.
- The finally block executed in all three calls, whether or not an exception was raised.
- The program did not terminate abnormally; each exception was handled by the matching except block.
- A user-defined exception is created by inheriting from the Exception class and is triggered using raise.

RESULT

Thus, the Python program for Exception Handling was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is an exception?**
   Ans: A runtime event that disrupts the normal flow of a program, e.g. ZeroDivisionError.
2. **What is the role of try, except, else and finally?**
   Ans: try holds risky code; except handles errors; else runs if no error occurred; finally always runs.
3. **What is the difference between a syntax error and an exception?**
   Ans: A syntax error is detected before execution; an exception occurs while the program is running.
4. **How do you raise an exception manually?**
   Ans: Using the raise keyword, e.g. raise ValueError('message').
5. **If there are multiple except blocks, which one executes?**
   Ans: The first block whose exception type matches; hence specific exceptions are placed before general ones.

EXPERIMENT 6

Recursion

AIM

To write and execute Python programs to understand the concept of Recursion and its operations.

ALGORITHM

1. Start.
2. Factorial: if n is 0 or 1 return 1 (base case); otherwise return n * factorial(n - 1).
3. Fibonacci: if n <= 1 return n (base case); otherwise return fibonacci(n - 1) + fibonacci(n - 2).
4. Tower of Hanoi: if n = 1, move the disk directly; otherwise move n - 1 disks to the auxiliary peg, move the largest disk to the destination, and move the n - 1 disks from the auxiliary peg to the destination.
5. Call the three functions and display the results.
6. Stop.

PROGRAM

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

def hanoi(n, source, auxiliary, destination):
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return
    hanoi(n - 1, source, destination, auxiliary)
    print(f"Move disk {n} from {source} to {destination}")
    hanoi(n - 1, auxiliary, source, destination)

print("Factorial of 5:", factorial(5))
print("Fibonacci series:", [fibonacci(i) for i in range(8)])
print("Tower of Hanoi (3 disks):")
hanoi(3, "A", "B", "C")
```

OUTPUT

```
Factorial of 5: 120
Fibonacci series: [0, 1, 1, 2, 3, 5, 8, 13]
Tower of Hanoi (3 disks):
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

OBSERVATION

- Each recursive function has a base case that stops the recursion and a recursive case that reduces the problem size.
- factorial(5) = 5 x 4 x 3 x 2 x 1 = 120.
- Tower of Hanoi with 3 disks required 7 moves, which agrees with the formula 2^n - 1.
- Every recursive call is placed on the call stack; very deep recursion raises RecursionError.

RESULT

Thus, the Python program for Recursion was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is recursion?**
   Ans: A technique in which a function calls itself to solve smaller instances of the same problem.
2. **What is a base case and why is it needed?**
   Ans: It is the condition that stops the recursion; without it the function calls itself endlessly.
3. **What happens when the recursion depth limit is exceeded?**
   Ans: Python raises RecursionError (default limit is about 1000 calls).
4. **What is the time complexity of the naive recursive Fibonacci function?**
   Ans: O(2^n); it can be reduced to O(n) using memoization.
5. **Compare recursion and iteration.**
   Ans: Recursion is shorter and natural for divide-and-conquer or tree problems but uses extra stack memory; iteration is usually faster and uses constant memory.

EXPERIMENT 7

Abstract Data Types (ADT) and User-Defined Types (UDT)

AIM

To write and execute Python programs to understand the concept of Abstract Data Types (ADT) and User-Defined Types (UDT) and its operations.

ALGORITHM

1. Start.
2. Create an abstract class Shape (the ADT) that declares the operations area() and perimeter() without implementing them.
3. Create user-defined classes Rectangle and Circle that inherit from Shape and implement both operations.
4. Try to create an object of Shape and handle the resulting TypeError.
5. Create objects of Rectangle and Circle and call the common operations.
6. Display the results.
7. Stop.

PROGRAM

```
from abc import ABC, abstractmethod

class Shape(ABC):                   # ADT: declares operations only
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):             # UDT implementing the ADT
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

class Circle(Shape):                # Another UDT
    PI = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return round(self.PI * self.radius ** 2, 2)

    def perimeter(self):
        return round(2 * self.PI * self.radius, 2)

try:
    shape = Shape()
except TypeError:
    print("Abstract class Shape cannot be instantiated.")

for shape in (Rectangle(5, 3), Circle(4)):
    print(type(shape).__name__, "- Area:", shape.area(),
          "| Perimeter:", shape.perimeter())
```

OUTPUT

```
Abstract class Shape cannot be instantiated.
Rectangle - Area: 15 | Perimeter: 16
Circle - Area: 50.27 | Perimeter: 25.13
```

OBSERVATION

- The ADT (Shape) specifies what operations are available, not how they work, and it cannot be instantiated.
- The UDTs (Rectangle and Circle) supply the implementation of the operations.
- The same calls, area() and perimeter(), behave differently for each type (polymorphism).
- Users of the type only need the interface; the internal data (length, radius) is hidden behind the methods (abstraction).

RESULT

Thus, the Python program for Abstract Data Types (ADT) and User-Defined Types (UDT) was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is an Abstract Data Type?**
   Ans: A logical model of data together with the operations on it, defined independent of its implementation.
2. **What is the difference between an ADT and a data structure?**
   Ans: An ADT describes what operations are provided; a data structure is the concrete way those operations are implemented.
3. **What is a User-Defined Type?**
   Ans: A type created by the programmer, such as a class, to model a real-world entity.
4. **How are abstract classes created in Python?**
   Ans: By inheriting from ABC (abc module) and marking methods with @abstractmethod.
5. **Give examples of ADTs.**
   Ans: List, Stack, Queue, Set, Map (Dictionary), Priority Queue.

EXPERIMENT 8

Stack Data Structure and its Operations

AIM

To write and execute Python programs to understand the concept of Stack and its operations.

ALGORITHM

1. Start.
2. Create an empty stack with a fixed capacity.
3. PUSH: if the stack is full, report overflow; otherwise add the element on top.
4. POP: if the stack is empty, report underflow; otherwise remove and return the top element.
5. PEEK: return the top element without removing it.
6. Display the stack and check is_empty() / size.
7. Stop.

PROGRAM

```
class Stack:
    def __init__(self, capacity=5):
        self.items = []
        self.capacity = capacity

    def is_empty(self):
        return len(self.items) == 0

    def is_full(self):
        return len(self.items) == self.capacity

    def push(self, item):
        if self.is_full():
            print("Stack Overflow! Cannot push", item)
        else:
            self.items.append(item)

    def pop(self):
        if self.is_empty():
            print("Stack Underflow!")
            return None
        return self.items.pop()

    def peek(self):
        return None if self.is_empty() else self.items[-1]

    def display(self):
        print("Stack (bottom -> top):", self.items)

s = Stack(capacity=4)
for x in (10, 20, 30, 40):
    s.push(x)
s.display()
s.push(50)
print("Peek:", s.peek())
print("Popped:", s.pop())
print("Popped:", s.pop())
s.display()
print("Size:", len(s.items), "| Empty?", s.is_empty())
```

OUTPUT

```
Stack (bottom -> top): [10, 20, 30, 40]
Stack Overflow! Cannot push 50
Peek: 40
Popped: 40
Popped: 30
Stack (bottom -> top): [10, 20]
Size: 2 | Empty? False
```

OBSERVATION

- The stack follows LIFO (Last In, First Out): 40, the last element pushed, was the first one popped.
- Pushing a fifth element into a stack of capacity 4 produced a Stack Overflow message.
- peek() returns the top element without changing the stack.
- Push, pop and peek each take O(1) time.

RESULT

Thus, the Python program for Stack was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is a stack?**
   Ans: A linear data structure in which insertion and deletion happen only at one end (the top), following LIFO.
2. **What are stack overflow and underflow?**
   Ans: Overflow: pushing onto a full stack. Underflow: popping from an empty stack.
3. **Mention applications of a stack.**
   Ans: Function call management, undo/redo, expression evaluation, balanced parentheses checking, browser back button, DFS.
4. **What is the time complexity of push and pop?**
   Ans: O(1) for both.
5. **How does a stack differ from a queue?**
   Ans: A stack is LIFO with one open end; a queue is FIFO with insertion at the rear and deletion at the front.

EXPERIMENT 9

Queue Data Structure and its Operations

AIM

To write and execute Python programs to understand the concept of Queue and its operations.

ALGORITHM

1. Start.
2. Create an empty queue with a fixed capacity.
3. ENQUEUE: if the queue is full, report overflow; otherwise insert the element at the rear.
4. DEQUEUE: if the queue is empty, report underflow; otherwise remove the element at the front.
5. Find the front and rear elements.
6. Display the queue and its size.
7. Stop.

PROGRAM

```
from collections import deque

class Queue:
    def __init__(self, capacity=4):
        self.items = deque()
        self.capacity = capacity

    def is_empty(self):
        return len(self.items) == 0

    def is_full(self):
        return len(self.items) == self.capacity

    def enqueue(self, item):
        if self.is_full():
            print("Queue Overflow! Cannot enqueue", item)
        else:
            self.items.append(item)

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow!")
            return None
        return self.items.popleft()

    def front(self):
        return None if self.is_empty() else self.items[0]

    def rear(self):
        return None if self.is_empty() else self.items[-1]

    def display(self):
        print("Queue (front -> rear):", list(self.items))

q = Queue()
for x in (5, 10, 15, 20):
    q.enqueue(x)
q.display()
q.enqueue(25)
print("Front:", q.front(), "| Rear:", q.rear())
print("Dequeued:", q.dequeue())
print("Dequeued:", q.dequeue())
q.display()
print("Size:", len(q.items))
```

OUTPUT

```
Queue (front -> rear): [5, 10, 15, 20]
Queue Overflow! Cannot enqueue 25
Front: 5 | Rear: 20
Dequeued: 5
Dequeued: 10
Queue (front -> rear): [15, 20]
Size: 2
```

OBSERVATION

- The queue follows FIFO (First In, First Out): 5, the first element inserted, was the first one removed.
- Enqueue at the rear and dequeue at the front each take O(1) time using collections.deque.
- Inserting a fifth element into a queue of capacity 4 produced a Queue Overflow message.
- Using list.pop(0) instead of deque.popleft() would take O(n) because all remaining elements shift.

RESULT

Thus, the Python program for Queue was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is a queue?**
   Ans: A linear data structure in which insertion is done at the rear and deletion at the front (FIFO).
2. **Name the types of queues.**
   Ans: Simple queue, circular queue, priority queue and double-ended queue (deque).
3. **Why is collections.deque preferred over a list for a queue?**
   Ans: deque gives O(1) operations at both ends, whereas list.pop(0) is O(n).
4. **What is the advantage of a circular queue?**
   Ans: It reuses freed space at the front, avoiding the wastage seen in a simple array-based queue.
5. **Mention applications of a queue.**
   Ans: CPU scheduling, printer spooling, BFS, buffering in I/O and request handling in servers.

EXPERIMENT 10

Singly Linked List and its Operations

AIM

To write and execute Python programs to understand the concept of Singly Linked List and its operations.

ALGORITHM

1. Start.
2. Define a Node with two fields: data and next.
3. Insert at beginning: create a node, set its next to head, and move head to the new node.
4. Insert at end: traverse to the last node and link the new node after it.
5. Delete: traverse while keeping a pointer to the previous node, then bypass the target node.
6. Search: traverse and compare each node's data with the key.
7. Reverse: traverse and reverse each next pointer, then update head.
8. Display the list after each operation.
9. Stop.

PROGRAM

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        node = Node(data)
        node.next = self.head
        self.head = node

    def insert_at_end(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = node

    def delete(self, key):
        temp, prev = self.head, None
        while temp and temp.data != key:
            prev, temp = temp, temp.next
        if temp is None:
            print(key, "not found")
            return
        if prev is None:
            self.head = temp.next
        else:
            prev.next = temp.next

    def search(self, key):
        temp, position = self.head, 0
        while temp:
            if temp.data == key:
                return position
            temp, position = temp.next, position + 1
        return -1

    def reverse(self):
        prev, current = None, self.head
        while current:
            nxt = current.next
            current.next = prev
            prev = current
            current = nxt
        self.head = prev

    def display(self):
        items, temp = [], self.head
        while temp:
            items.append(str(temp.data))
            temp = temp.next
        print(" -> ".join(items) + " -> None")

ll = SinglyLinkedList()
ll.insert_at_end(20)
ll.insert_at_end(30)
ll.insert_at_beginning(10)
ll.insert_at_end(40)
print("Linked list:")
ll.display()
print("Position of 30 (0-based):", ll.search(30))
ll.delete(20)
print("After deleting 20:")
```

```
ll.display()
ll.reverse()
print("After reversing:")
ll.display()
```

OUTPUT

```
Linked list:
10 -> 20 -> 30 -> 40 -> None
Position of 30 (0-based): 2
After deleting 20:
10 -> 30 -> 40 -> None
After reversing:
40 -> 30 -> 10 -> None
```

OBSERVATION

- Each node stores data and a reference to the next node; the last node points to None.
- Insertion at the beginning is O(1), whereas insertion at the end is O(n) because the list must be traversed.
- Deleting 20 required only a change of one pointer; no elements were shifted (unlike an array).
- Reversing the list changes only the next pointers and takes O(n) time with O(1) extra space.

RESULT

Thus, the Python program for Singly Linked List was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What are the components of a node in a singly linked list?**
   Ans: A data field and a next pointer (reference) to the following node.
2. **What are the advantages of a linked list over an array?**
   Ans: Dynamic size and O(1) insertion/deletion at a known position without shifting elements.
3. **What is the time complexity of inserting at the end without a tail pointer?**
   Ans: O(n); with a tail pointer it is O(1).
4. **How can the middle node of a linked list be found in one pass?**
   Ans: Use two pointers: a slow pointer moving one step and a fast pointer moving two steps.
5. **How can a cycle in a linked list be detected?**
   Ans: Floyd's cycle detection (tortoise and hare) algorithm.

EXPERIMENT 11

Doubly Linked List and its Operations

AIM

To write and execute Python programs to understand the concept of Doubly Linked List and its operations.

ALGORITHM

1. Start.
2. Define a Node with three fields: data, prev and next.
3. Maintain two references, head and tail.
4. Insert at beginning: link the new node before head and update head and the prev pointer of the old head.
5. Insert at end: link the new node after tail and update tail.
6. Delete: locate the node, connect its previous and next nodes to each other, and update head or tail if required.
7. Traverse forward from head and backward from tail to display the list.
8. Stop.

PROGRAM

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def insert_at_beginning(self, data):
        node = Node(data)
        if self.head is None:
            self.head = self.tail = node
        else:
            node.next = self.head
            self.head.prev = node
            self.head = node

    def insert_at_end(self, data):
        node = Node(data)
        if self.tail is None:
            self.head = self.tail = node
        else:
            node.prev = self.tail
            self.tail.next = node
            self.tail = node

    def delete(self, key):
        temp = self.head
        while temp and temp.data != key:
            temp = temp.next
        if temp is None:
            print(key, "not found")
            return
        if temp.prev:
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next:
            temp.next.prev = temp.prev
        else:
            self.tail = temp.prev

    def display_forward(self):
        items, temp = [], self.head
        while temp:
            items.append(str(temp.data))
            temp = temp.next
        print("Forward :", " <-> ".join(items))

    def display_backward(self):
        items, temp = [], self.tail
        while temp:
            items.append(str(temp.data))
            temp = temp.prev
        print("Backward:", " <-> ".join(items))

dll = DoublyLinkedList()
for x in (20, 30, 40):
    dll.insert_at_end(x)
dll.insert_at_beginning(10)
dll.display_forward()
dll.display_backward()

dll.delete(30)
print("After deleting 30:")
dll.display_forward()
dll.display_backward()
```

OUTPUT

```
Forward : 10 <-> 20 <-> 30 <-> 40
Backward: 40 <-> 30 <-> 20 <-> 10
After deleting 30:
Forward : 10 <-> 20 <-> 40
Backward: 40 <-> 20 <-> 10
```

OBSERVATION

- Each node holds references to both its previous and next nodes, so the list can be traversed in both directions.
- The forward and backward outputs are exact reverses of each other, confirming that the prev and next links are consistent.
- Deleting a node whose address is known is O(1), because the previous node is reachable through prev.
- A doubly linked list needs extra memory for the prev pointer of every node.

RESULT

Thus, the Python program for Doubly Linked List was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **How does a doubly linked list differ from a singly linked list?**
   Ans: Each node has an additional prev pointer, which allows traversal in both directions.
2. **What are the advantages of a doubly linked list?**
   Ans: Bidirectional traversal and O(1) deletion of a given node without tracking the previous node.
3. **What are its disadvantages?**
   Ans: More memory per node and more pointers to update during insertion and deletion.
4. **What is a circular doubly linked list?**
   Ans: A doubly linked list where the last node's next points to the head and the head's prev points to the last node.
5. **Mention applications of a doubly linked list.**
   Ans: Browser history (back/forward), undo/redo, music playlists and LRU cache implementation.

EXPERIMENT 12

Tree Data Structure and its Operations

AIM

To write and execute Python programs to understand the concept of Tree (Binary Search Tree) and its operations.

ALGORITHM

1. Start.
2. Define a Node with fields key, left and right.
3. Insert: starting at the root, go left if key < node, right if key > node, and place the new node at the first empty position.
4. Search: compare the key with the current node and move left or right until it is found or None is reached.
5. Height: height(node) = 1 + max(height(left), height(right)); height of an empty tree is 0.
6. Count nodes recursively; find the minimum (leftmost) and maximum (rightmost) key.
7. Display the results.
8. Stop.

PROGRAM

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert(self.root, key)

    def _insert(self, node, key):
        if node is None:
            return Node(key)
        if key < node.key:
            node.left = self._insert(node.left, key)
        elif key > node.key:
            node.right = self._insert(node.right, key)
        return node

    def search(self, key):
        node = self.root
        while node:
            if key == node.key:
                return True
            node = node.left if key < node.key else node.right
        return False

    def height(self, node):
        if node is None:
            return 0
        return 1 + max(self.height(node.left), self.height(node.right))

    def count(self, node):
        if node is None:
            return 0
        return 1 + self.count(node.left) + self.count(node.right)

    def minimum(self):
        node = self.root
        while node.left:
            node = node.left
        return node.key

    def maximum(self):
        node = self.root
        while node.right:
            node = node.right
        return node.key

tree = BinarySearchTree()
for key in (50, 30, 70, 20, 40, 60, 80):
    tree.insert(key)

print("Root:", tree.root.key)
print("Search 40:", tree.search(40))
print("Search 90:", tree.search(90))
print("Height:", tree.height(tree.root))
print("Number of nodes:", tree.count(tree.root))
print("Minimum:", tree.minimum(), "| Maximum:", tree.maximum())
```

OUTPUT

```
Root: 50
Search 40: True
Search 90: False
Height: 3
Number of nodes: 7
Minimum: 20 | Maximum: 80
```

OBSERVATION

- The tree built from 50, 30, 70, 20, 40, 60, 80 has 50 as the root; 30 and 70 are its children; 20, 40, 60 and 80 are leaves.
- Smaller keys are stored in the left subtree and larger keys in the right subtree (BST property).
- The minimum key is the leftmost node and the maximum key is the rightmost node.
- Search, insert and height operations follow a single path, so their time is O(h), where h is the height of the tree.

RESULT

Thus, the Python program for Tree (Binary Search Tree) was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What is a tree? Define root, leaf and height.**
   Ans: A non-linear hierarchical structure of nodes. Root: topmost node; leaf: node with no children; height: number of levels (or edges) on the longest root-to-leaf path.
2. **What is the property of a Binary Search Tree?**
   Ans: For every node, all keys in the left subtree are smaller and all keys in the right subtree are larger.
3. **What is the time complexity of searching in a BST?**
   Ans: O(log n) on average (balanced) and O(n) in the worst case (skewed tree).
4. **What is the difference between a binary tree and a BST?**
   Ans: A binary tree allows at most two children per node with no ordering rule; a BST additionally keeps its keys ordered.
5. **What is a balanced tree and why is it needed?**
   Ans: A tree whose height is kept close to log n (e.g. AVL, Red-Black), guaranteeing O(log n) operations.

EXPERIMENT 13

Tree Traversals

AIM

To write and execute Python programs to understand the concept of Tree Traversals and its operations.

ALGORITHM

1. Start.
2. Build the binary search tree with keys 50, 30, 70, 20, 40, 60, 80.
3. Inorder (Left, Root, Right): traverse the left subtree, visit the root, traverse the right subtree.
4. Preorder (Root, Left, Right): visit the root, traverse the left subtree, traverse the right subtree.
5. Postorder (Left, Right, Root): traverse the left subtree, traverse the right subtree, visit the root.
6. Level order: use a queue; remove a node, visit it, and add its left and right children to the queue.
7. Display each traversal.
8. Stop.

PROGRAM

```
from collections import deque

class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root

def inorder(node):
    return inorder(node.left) + [node.key] + inorder(node.right) if node else []

def preorder(node):
    return [node.key] + preorder(node.left) + preorder(node.right) if node else []

def postorder(node):
    return postorder(node.left) + postorder(node.right) + [node.key] if node else []

def level_order(root):
    result, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        result.append(node.key)
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return result

root = None
for key in (50, 30, 70, 20, 40, 60, 80):
    root = insert(root, key)

print("Inorder    :", inorder(root))
print("Preorder   :", preorder(root))
print("Postorder  :", postorder(root))
print("Level order:", level_order(root))
```

OUTPUT

```
Inorder    : [20, 30, 40, 50, 60, 70, 80]
Preorder   : [50, 30, 20, 40, 70, 60, 80]
Postorder  : [20, 40, 30, 60, 80, 70, 50]
Level order: [50, 30, 70, 20, 40, 60, 80]
```

OBSERVATION

- Inorder traversal of a BST prints the keys in ascending order: 20, 30, 40, 50, 60, 70, 80.
- Preorder visits the root first (50), so it is used to copy a tree; postorder visits the root last (50), so it is used to delete a tree.
- Level order visits the nodes level by level using a queue (breadth-first), whereas the other three are depth-first.
- Every traversal visits each node exactly once, so the time complexity is O(n).

RESULT

Thus, the Python program for Tree Traversals was written, executed successfully and the output was verified.

VIVA QUESTIONS

1. **What are the types of tree traversal?**
   Ans: Depth-first: inorder, preorder, postorder. Breadth-first: level order.
2. **What does the inorder traversal of a BST give?**
   Ans: The keys in sorted (ascending) order.
3. **Which traversal is used to create a copy of a tree, and which to delete a tree?**
   Ans: Preorder to copy; postorder to delete (children are removed before the parent).
4. **What is the difference between DFS and BFS traversals?**
   Ans: DFS goes as deep as possible before backtracking (uses recursion/stack); BFS visits level by level (uses a queue).
5. **What is the time and space complexity of a traversal?**
   Ans: Time O(n); space O(h) for recursive DFS (h = height) and O(w) for level order (w = maximum width).